

# Node Initializr

Documentacao tecnica da Sprint 1 - Fundacao do monorepo e dominio inicial

Data: 19/06/2026

Repositorio: C:\Users\User\Desktop\node-initializr

Este documento registra o que foi construido na Sprint 1, por que cada artefato existe, como os modulos se conectam e quais comandos validam o estado atual do projeto.

# 1. Visao Executiva

A Sprint 1 estabeleceu a base tecnica do Node Initializr: um monorepo pnpm/Turborepo com frontend React, backend Express, pacote compartilhado de contrato, validacao de payload e resolver inicial de fragmentos. O foco foi preparar um alicerce testavel antes da geracao real de arquivos e zip.

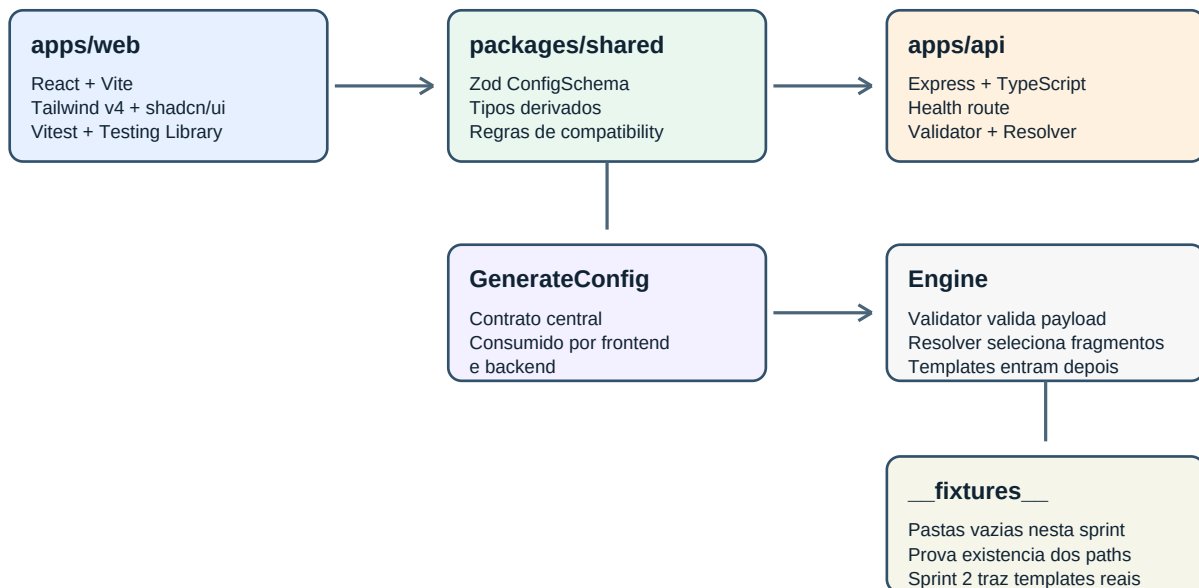
- Monorepo com workspaces em apps/\* e packages/\*.
- Frontend Vite + React + Tailwind v4 + shadcn/ui com Vitest e Testing Library.
- Backend Express + TypeScript com rota GET /api/health.
- Pacote @node-initializr/shared com ConfigSchema, tipos derivados e regras de compatibility.
- Validator backend usando o schema compartilhado e envelope de erro padronizado.
- Template Resolver inicial retornando Fragment[] ordenado por prioridade e baseado em fixtures.
- Ciclo TDD aplicado em Validator, Resolver e Compatibility frontend.

## Status da sprint

| Item                 | Status                                                                  |
|----------------------|-------------------------------------------------------------------------|
| NI-001 a NI-013      | Concluidas                                                              |
| Testes automatizados | Passando em api e web; shared com teste no-op ate haver testes proprios |
| Build                | Passando nos tres workspaces                                            |
| Lint e formatacao    | Passando apos ajustes de ESLint/Prettier                                |

## 2. Arquitetura Atual

A arquitetura segue uma divisao simples: apps/web cuida da experiencia do usuario, apps/api cuida da API e do motor de geracao, e packages/shared concentra o contrato de dados comum.



### Fluxo alvo do produto

```
Usuario preenche formulario no React
-> frontend valida compatibility localmente
-> POST /api/generate envia GenerateConfig
-> API valida com Validator
-> TemplateResolver seleciona fragmentos
-> composers renderizam arquivos e package.json
-> backend retorna .zip
```

Na Sprint 1, o endpoint de geracao ainda nao foi implementado. O trabalho finalizado cobre o contrato, as validacoes, a estrutura de apps e o resolver inicial que sera consumido pelos composers nas proximas sprints.

### 3. Monorepo e Ferramentas

| Artefato            | O que faz                                                               | Por que existe                                                                                               |
|---------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| package.json raiz   | Define scripts dev/build/test/lint/format e packageManager pnpm@9.15.0. | Centraliza comandos do monorepo e permite o Turborepo orquestrar os workspaces.                              |
| pnpm-workspace.yaml | Declara apps/* e packages/*.                                            | Faz o pnpm reconhecer API, web e shared como pacotes do mesmo workspace.                                     |
| turbo.json          | Configura tasks build, lint, test e dev.                                | Permite executar validacoes por pacote, com dependencias entre workspaces.                                   |
| .nvmrc              | Declara Node 20.                                                        | Fixa a versao LTS prevista no plano.                                                                         |
| .gitignore          | Ignora node_modules, dist, .turbo e .env.                               | Evita versionar artefatos locais, build e secrets.                                                           |
| .prettierrc         | Define estilo de formatacao.                                            | Garante consistencia automatica entre pacotes.                                                               |
| eslint.config.mjs   | Configura regras base TypeScript/JavaScript.                            | Fornece lint compartilhado para api e shared; import/order foi removido por incompatibilidade com ESLint 10. |

#### Scripts raiz principais

```
pnpm dev -> turbo dev
pnpm build -> turbo build
pnpm test -> turbo test
pnpm lint -> turbo lint
pnpm format -> prettier . --write
pnpm format:check -> prettier . --check
```

## 4. packages/shared

O pacote `@node-initializr/shared` é a fonte única de verdade para o payload de geração. Ele evita duplicação de schema entre frontend e backend.

| Arquivo              | Responsabilidade                                                                                        |
|----------------------|---------------------------------------------------------------------------------------------------------|
| src/schema.ts        | Define enums e ConfigSchema com Zod, incluindo regras de compatibilidade via <code>superRefine</code> . |
| src/types.ts         | Deriva <code>GenerateConfig</code> e tipos literais a partir do schema.                                 |
| src/compatibility.ts | Exporta <code>checkCompatibility(config)</code> , uma função pura usada pelo frontend.                  |
| src/index.ts         | Reexporta schema, types e compatibility para consumo por <code>@node-initializr/shared</code> .         |

### Contrato GenerateConfig

```
name: kebab-case, 1 a 64 caracteres
packageManager: npm | pnpm | yarn
framework: express | fastify | nestjs | hono
language: typescript | javascript
architecture: modular | clean | mvc
database: postgresql | mysql | mongodb | sqlite | none
orm: prisma | typeorm | drizzle | mongoose | none
auth: jwt | clerk | none
messaging: none | rabbitmq | bullmq
dependencies: redis | swagger | jest | docker | github-actions | eslint | pino | winston
```

### Regras de compatibilidade

- ORM diferente de none exige database diferente de none.
- Mongoose é exclusivo para MongoDB.
- Drizzle não suporta MongoDB.
- BullMQ exige Redis em dependencies.

## 5. apps/api

A API foi inicializada com Express e TypeScript. Nesta sprint ela expoe apenas health check e o nucleo de dominio usado futuramente pelo endpoint POST /api/generate.

| Artefato                   | O que faz                                                             | Por que existe                                                       |
|----------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------|
| src/app.ts                 | Cria a instancia Express, habilita JSON e monta healthRouter em /api. | Separa configuracao HTTP do listen, permitindo testes com Supertest. |
| src/server.ts              | Inicializa app.listen usando PORT ou 3000.                            | Isola o processo runtime da aplicacao testavel.                      |
| src/routes/health.route.ts | GET /api/health retorna { status: 'ok' }.                             | Prova que a API sobe e responde antes de rotas de geracao.           |
| vitest.config.ts           | Configura ambiente node.                                              | Permite testes unitarios/integracao da API.                          |

### Health check

```
GET /api/health -> 200 OK
Body: { "status": "ok" }
```

### Validator

O Validator recebe unknown, executa ConfigSchema.safeParse e retorna uma uniao discriminada. Quando valido, retorna success true com GenerateConfig tipado. Quando invalido, retorna success false com ValidationError contendo code, message, field e details.

```
{ success: true, data: GenerateConfig }
{ success: false, error: ValidationError }
```

Erros estruturais usam VALIDATION\_ERROR. Violacoes conhecidas de compatibilidade usam INVALID\_COMBINATION. As mensagens foram extraidas para validator.messages.ts durante o refactor.

## 6. Template Resolver

O Template Resolver mapeia uma configuracao valida para uma lista ordenada de fragmentos. Ele ainda nao renderiza arquivos; apenas decide quais fragmentos serao aplicados nas proximas etapas do motor.

| Arquivo                        | Responsabilidade                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------|
| src/engine/types.ts            | Define Fragment, FragmentManifest, EnvVar e reexporta GenerateConfig.                        |
| src/engine/fragmentRegistry.ts | Mapeia ids de fragmento para path, priority e manifest.                                      |
| src/engine/resolver.ts         | Seleciona base, framework, ORM, auth e dependencies, depois ordena por priority.             |
| src/engine/__fixtures__/*      | Pastas vazias que provam existencia dos paths enquanto os templates reais ainda nao existem. |
| src/engine/resolver.test.ts    | Cobre base, framework, ORM, auth, dependencies, ordenacao e integridade.                     |

### Prioridades atuais

| Categoria   | Exemplos                                                                   | Priority |
|-------------|----------------------------------------------------------------------------|----------|
| Base        | base/typescript, base/javascript                                           | 10       |
| Framework   | frameworks/express, frameworks/fastify, frameworks/nestjs, frameworks/hono | 20       |
| ORM         | orms/prisma, orms/typeorm, orms/drizzle, orms/mongoose                     | 30       |
| Auth        | auth/jwt, auth/clerk                                                       | 40       |
| Deps gerais | deps/redis, deps/swagger, deps/jest, deps/eslint, deps/pino, deps/winston  | 50       |
| Infra       | deps/docker, deps/github-actions                                           | 90       |

As fixtures estarem vazias e esperado nesta sprint. A spec exige que o path exista no disco; conteudo de templates entra depois, quando FileRenderer e templates reais forem implementados.

## 7. apps/web

O frontend foi criado com Vite, React, TypeScript, Tailwind CSS v4 e shadcn/ui. A UI ainda e a tela base do Vite, mas o ambiente de teste, build, estilos e componentes ja esta pronto.

| Artefato                     | O que faz                                                | Por que existe                                                      |
|------------------------------|----------------------------------------------------------|---------------------------------------------------------------------|
| vite.config.ts               | Configura React, Tailwind v4, paths e Vitest jsdom.      | Unifica dev server, build e testes do frontend.                     |
| components.json              | Configura shadcn/ui com Radix Nova, lucide e aliases @/. | Permite adicionar componentes UI padronizados.                      |
| src/test/setup.ts            | Importa @testing-library/jest-dom.                       | Adiciona matchers como toBeInTheDocument.                           |
| src/App.test.tsx             | Smoke test do App.                                       | Prova que o setup React/Vitest/jsdom funciona.                      |
| src/components/ui/button.tsx | Primeiro componente shadcn.                              | Valida aliases e dependencias de UI.                                |
| src/lib/utils.ts             | Funcao cn com clsx e tailwind-merge.                     | Suporte padrao para composicao de classes shadcn.                   |
| src/lib/compatibility.ts     | Reexporta checkCompatibility do shared.                  | Mantem frontend consumindo a mesma regra do contrato compartilhado. |

### Compatibility no frontend

A NI-012 criou os testes RED de compatibility no frontend. A NI-013 implementou o GREEN delegando para @node-initializr/shared. Assim, as regras de combinacao ficam em um unico lugar.

## 8. Cobertura de Testes e TDD

A sprint seguiu o ciclo RED -> GREEN -> REFACTOR nas partes de dominio. Primeiro foram criados testes que falhavam, depois a implementacao minima, e por fim refactors pequenos sem mudar comportamento.

| Area              | Arquivo de teste                         | O que cobre                                                                                       |
|-------------------|------------------------------------------|---------------------------------------------------------------------------------------------------|
| API health        | apps/api/src/routes/health.route.test.ts | GET /api/health com status 200 e body { status: 'ok' }.                                           |
| Validator         | apps/api/src/engine/validator.test.ts    | Payload valido, campos obrigatorios, compatibilidade ORM/database, messaging e estrutura de erro. |
| Resolver          | apps/api/src/engine/resolver.test.ts     | Fragmento base, framework, ORM, auth, dependencies, ordenacao e paths existentes.                 |
| Web App           | apps/web/src/App.test.tsx                | Smoke test de renderizacao em jsdom.                                                              |
| Compatibility web | apps/web/src/lib/compatibility.test.ts   | Sem erros, erros de ORM, erro BullMQ/Redis e multiplas violacoes.                                 |

### Comandos de validacao usados

```
pnpm format:check
pnpm lint
pnpm build
pnpm test

Validacoes por pacote:
pnpm test --filter api
pnpm build --filter api
pnpm lint --filter api
pnpm test --filter web
pnpm build --filter web
pnpm lint --filter web
pnpm build --filter @node-initializr/shared
pnpm lint --filter @node-initializr/shared
```

## 9. Decisoes Tecnicas e Ajustes Feitos

| Decisao                               | Motivo                                                                                                   |
|---------------------------------------|----------------------------------------------------------------------------------------------------------|
| Usar pnpm workspaces + Turborepo      | Atende ao plano da sprint e prepara execucao por pacote com dependencias internas.                       |
| Separar packages/shared               | Evita divergencia de schema entre frontend e backend.                                                    |
| Usar NodeNext no backend/shared       | Alinha o projeto com ESM moderno e imports .js em arquivos TypeScript.                                   |
| Manter templates como fixtures vazias | Cumpre a spec do resolver sem antecipar FileRenderer ou templates reais.                                 |
| Reexportar compatibility no web       | Evita duplicacao de regra de negocio no frontend.                                                        |
| Remover import/order por enquanto     | eslint-plugin-import 2.32 quebrou com ESLint 10; manter a regra bloquearia progresso sem ganho imediato. |
| Usar script test no-op em shared      | Shared ainda nao tem testes proprios nesta sprint; isso permite pnpm test global passar.                 |
| Configurar .vscode/settings.json      | Evita falsos positivos do editor em monorepo com multiplos tsconfigs.                                    |

### Observacoes tecnicas

- O warning do Turbo sobre no output files found em builds noEmit nao indica falha; apenas mostra que a task nao gera dist.
- O aviso do Vite sobre vite-tsconfig-paths indica que Vite ja tem resolve.tsconfigPaths nativo; pode ser simplificado depois.
- O pacote shared usa teste no-op temporario. Quando houver testes de schema/compatibility no shared, trocar para vitest run.
- Revisar imports relativos do shared para manter consistencia NodeNext caso algum editor ou build futuro acuse divergencia.

## 10. Mapa de Artefatos

| Caminho                                   | Descricao                                               |
|-------------------------------------------|---------------------------------------------------------|
| apps/api/src/app.ts                       | Instancia Express testavel sem listen.                  |
| apps/api/src/server.ts                    | Entrada runtime da API.                                 |
| apps/api/src/routes/health.route.ts       | Health check.                                           |
| apps/api/src/engine/validator.ts          | Valida payload e padroniza erro.                        |
| apps/api/src/engine/validator.messages.ts | Centraliza mensagens e lista de erros de compatibility. |
| apps/api/src/engine/resolver.ts           | Mapeia GenerateConfig para Fragment[].                  |
| apps/api/src/engine/fragmentRegistry.ts   | Registry de fragmentos e priorities.                    |
| apps/api/src/engine/types.ts              | Tipos de engine.                                        |
| apps/web/src/lib/compatibility.ts         | Ponte do frontend para rules compartilhadas.            |
| apps/web/src/components/ui/button.tsx     | Componente shadcn inicial.                              |
| packages/shared/src/schema.ts             | Schema Zod central.                                     |
| packages/shared/src/types.ts              | Tipos derivados.                                        |
| packages/shared/src/compatibility.ts      | Regras puras de compatibility.                          |

## 11. Proximos Passos

A Sprint 1 encerra com fundacao pronta. A sequencia natural e iniciar a Sprint 2 com templates reais, composers e a rota de geracao.

- Implementar PackageComposer com merge de dependencies, devDependencies e scripts.
- Implementar FileRenderer para renderizar templates EJS em GeneratedFile[].
- Substituir fixtures vazias por templates reais conforme o registry.
- Implementar POST /api/generate com Validator, Resolver, composers e zip.
- Adicionar smoke tests do projeto gerado quando templates reais existirem.
- Revisitar import/order quando houver versao compativel com ESLint 10 ou fixar ESLint 9.
- Remover vite-tsconfig-paths usando resolve.tsconfigPaths nativo do Vite, se desejado.

### Resumo final

O projeto agora possui uma base limpa, modular e testada. Qualquer pessoa que entre no repositorio consegue identificar onde fica o contrato de configuracao, como a API valida payloads, como o resolver escolhe fragmentos e como o frontend reutiliza as mesmas regras.